

Optimierung II: Verfahren 2. Ordnung, Stochastischer GD, Nebenbedingungen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Aus Optimierung I:**

- *Gradient Descent* (Gradientenabstieg, GD): Iterationsvorschrift, Schrittweite, Konvergenzraten
- *Newton-Raphson-Verfahren* für nichtlineare Gleichungen $f(x) = 0$
- Optimalitätsbedingungen: $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$, Definitheit von $\mathbf{H}_f(\mathbf{x}^*)$
- *Lipschitz-Konstante* und *Konditionszahl* $\kappa_f = L/\mu$

- **Aus Konvexität:**

- *Konvexe Mengen und Funktionen*; lokale = globale Minima

- **Lineare Algebra:** *positiv definite* Matrizen, Eigenwerte

- **Analysis:** Taylor-Approximation 1. und 2. Ordnung

Opener-Quiz: Was wissen Sie noch?

Frage 1: Wie lautet die Iterationsvorschrift des *Gradientenabstiegs*?

1. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \gamma \nabla f(\mathbf{x}^{(k)})^\top$
2. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top$
3. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma f(\mathbf{x}^{(k)})$
4. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \nabla f(\mathbf{x}^{(k)})^\top$

Frage 2: Sei $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ und $\mathbf{H}_f(\mathbf{x}^*)$ *indefinit*. Dann ist $\mathbf{x}^*$...

1. ein lokales Minimum
2. ein lokales Maximum
3. ein Sattelpunkt
4. ein globales Minimum

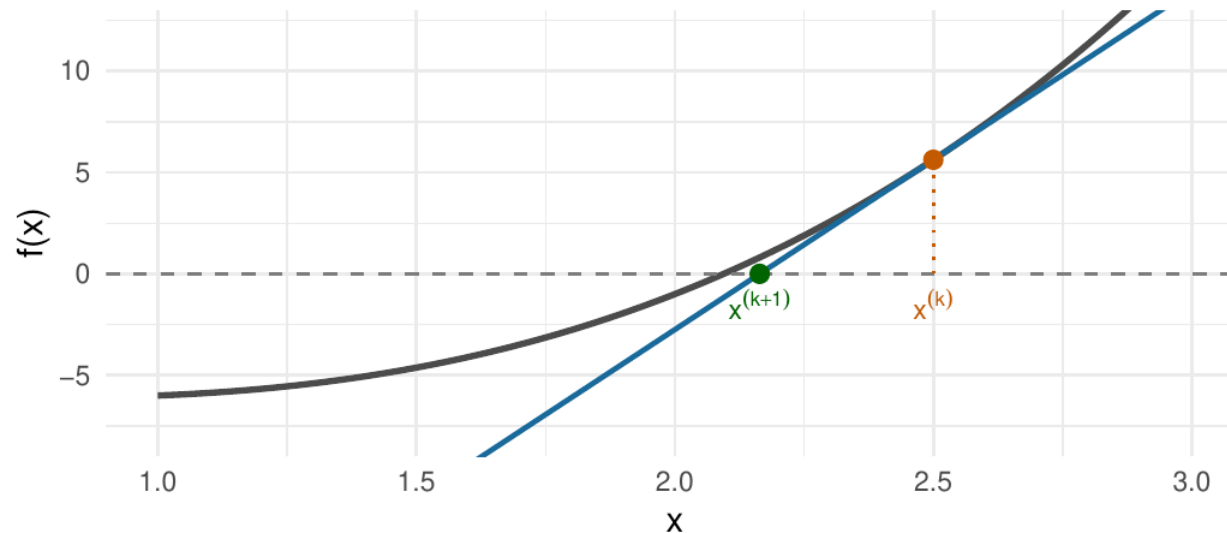
Brücke: Von Newton-Raphson zur Newton-Optimierung

Letzte Woche:

Newton-Raphson löst $f(x) = 0$ über die Nullstelle der Tangente:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Newton-Raphson: Tangente → Nullstelle



Heute:

Am Minimum gilt $\nabla f(x^*) = \mathbf{0}^\top$.

⇒ **Newton-Optimierung** = Newton-Raphson, angewandt auf die Gleichung $\nabla f(x) = \mathbf{0}^\top$!

Newton-Verfahren

Newton-Verfahren: Idee & Schritt

Problem bei GD:

Berücksichtigt nur die *Steigung*, nicht die *Krümmung*

→ approximiert f lokal durch Tangente (Taylor 1. Ordnung)

Newton-Idee:

Approximiere f lokal durch eine *Parabel* (Taylor 2. Ordnung):

$$f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})\mathbf{h} + \frac{1}{2}\mathbf{h}^\top \mathbf{H}_f(\mathbf{x})\mathbf{h}$$

→ Minimiere diese Approximation bzgl. $\mathbf{h}$:

$$\nabla_{\mathbf{h}} [\dots] = \nabla f(\mathbf{x}) + \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \stackrel{!}{=} \mathbf{0}^\top \quad \Longleftrightarrow \quad \mathbf{h} = -\mathbf{H}_f(\mathbf{x})^{-1} \nabla f(\mathbf{x})^\top$$

Newton-Schritt

In Iteration k wird der nächste Punkt $\mathbf{x}^{(k+1)}$ berechnet als:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \nabla f(\mathbf{x}^{(k)})^\top$$

- **Voraussetzung:** $\mathbf{H}_f(\mathbf{x}^{(k)})$ muss invertierbar sein (Konvexität liefert nur $\mathbf{H}_f \succeq 0$!)
- Abbruchkriterien wie bei *Gradient Descent*
- In der Praxis löst man das LGS, statt $\mathbf{H}_f^{-1}$ explizit zu berechnen
— **Invertiere niemals eine Matrix!**

Verbindung: Newton-Raphson $\leftrightarrow$ Newton-Optimierung

Nullstellensuche	Optimierung
Löse $f(x) = 0$	Löse $\nabla f(\mathbf{x}) = \mathbf{0}^\top$
$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$	$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f^{-1} \nabla f(\mathbf{x}^{(k)})^\top$

$\implies$

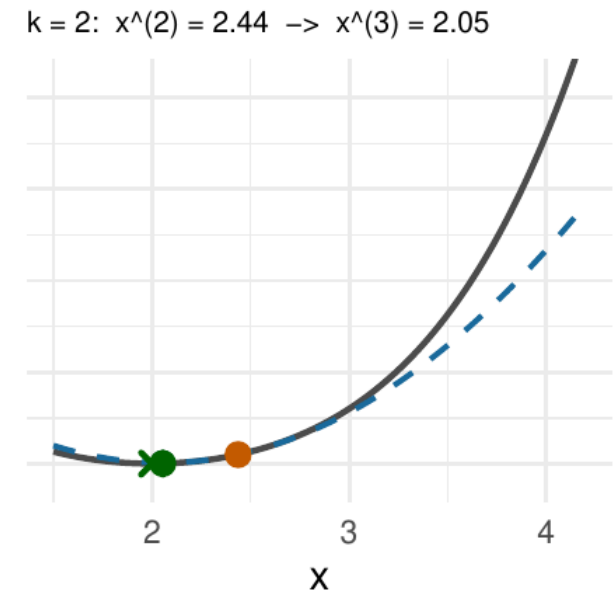
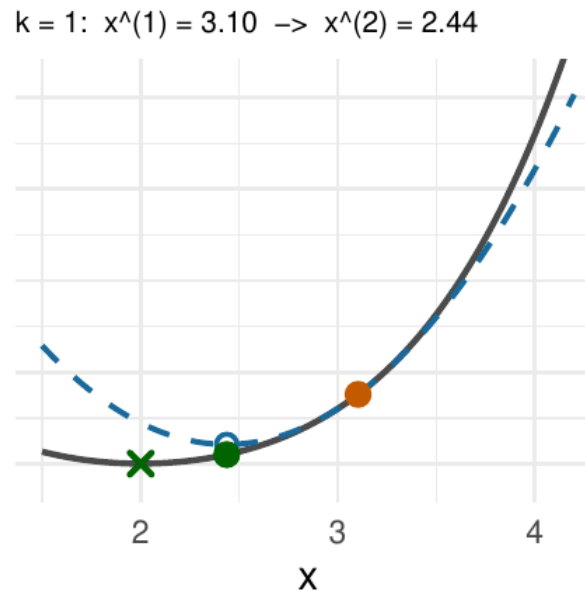
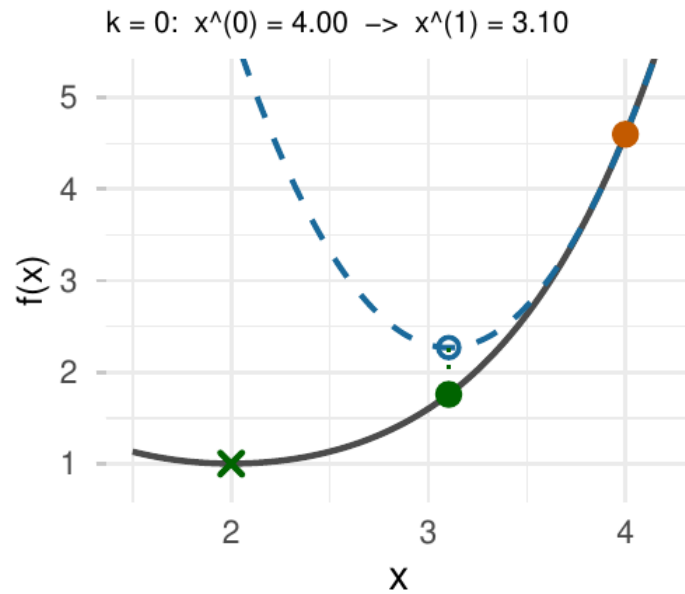
Newton für Optimierung $\simeq$ Newton-Raphson für $g(\mathbf{x}) := \nabla f(\mathbf{x})^\top = \mathbf{0}$

- Jacobi-Matrix von g : $\mathbf{J}_g = \mathbf{H}_f$
- Newton-Raphson:

$$\begin{aligned}\mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} - \mathbf{J}_g^{-1}(\mathbf{x}^{(k)}) g(\mathbf{x}^{(k)}) \\ &= \mathbf{x}^{(k)} - \mathbf{H}_f^{-1}(\mathbf{x}^{(k)}) \nabla f(\mathbf{x}^{(k)})^\top \quad \checkmark\end{aligned}$$

Newton-Verfahren: Geometrische Intuition

Newton-Iterationen für die konvexe Funktion $f(x) = 0.5(x-2)^2 + 0.1(x-2)^4 + 1$



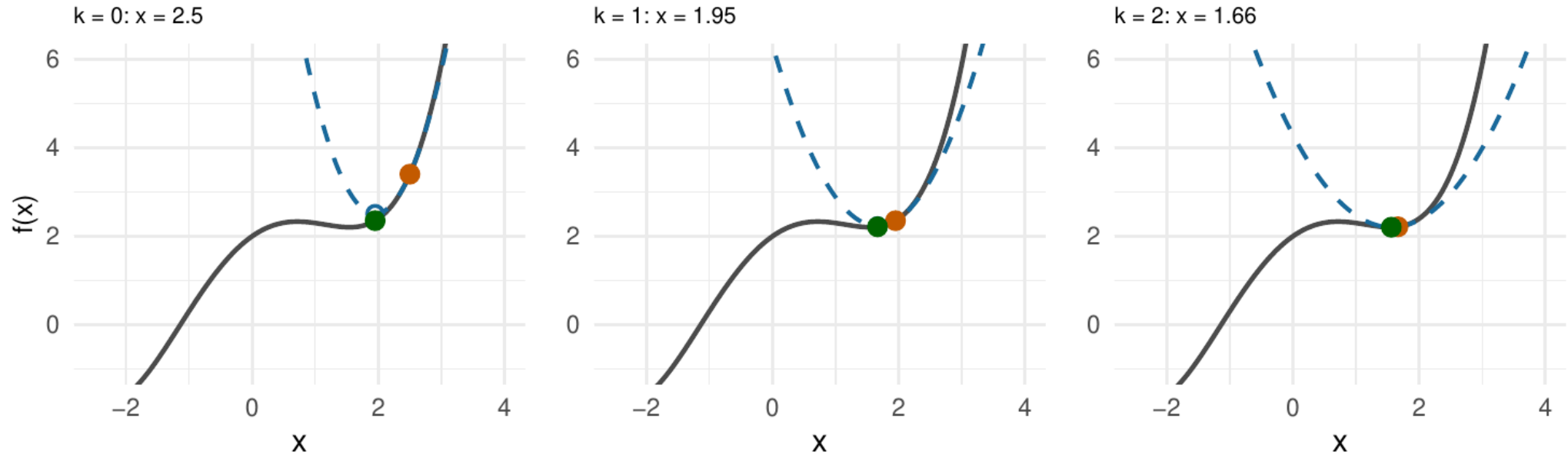
orange: aktueller Punkt

blau gestrichelt: quadratische Taylor-Approximation T_2

grün: nächster Iterationspunkt.

Newton für nicht-konvexe Funktionen

Newton für $f(x) = 0.1x^4 - 0.8x^2 + x + 2$ (nicht-konvex)



Bei nicht-konvexen Funktionen kann Newton auch zu beliebigen **lokalen** oder globalen kritischen Punkten — also auch Sattelpunkten oder *Maxima* — konvergieren.

Quasi-Newton-Verfahren

Quasi-Newton-Verfahren

Problem:

Hesse-Matrix $\mathbf{H}_f(\mathbf{x})$ ist teuer zu berechnen, zu speichern ($O(n^2)!$) und zu invertieren ($O(n^3)!$).

Idee:

Approximiere $\mathbf{H}_f^{-1}$ iterativ aus Gradienten-Informationen!

Quasi-Newton-Update

Der nächste Punkt $\mathbf{x}^{(k+1)}$ wird berechnet als:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma_k \mathbf{B}_k \nabla f(\mathbf{x}^{(k)})^\top,$$

wobei $\mathbf{B}_k \approx \mathbf{H}_f^{-1}(\mathbf{x}^{(k)})$ aus vorherigen Schritten geschätzt wird und γ_k aus einer *Line Search* stammt.

- Vermeidet Invertieren von $\mathbf{H}_f$
- Oft *Low-Rank-Updates* von $\mathbf{B}_k$ (BFGS: Rang 2) oder Diagonal-Approximationen
- Schrittweite γ_k kommt automatisch aus einer *Line Search* (ohne sie kann schon der erste Schritt aus $\mathbf{B}_0 = \mathbf{I}$ den Funktionswert erhöhen)

BFGS-Verfahren

BFGS = Broyden–Fletcher–Goldfarb–Shanno

Aktualisiert $\mathbf{B}_k$ so, dass die **Sekantenbedingung** $f''(x) \approx \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k}$ erfüllt ist:

$$\mathbf{B}_{k+1} \underbrace{\left(\nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}) \right)}_{\mathbf{y}_k}^\top = \underbrace{\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}}_{\mathbf{s}_k}$$

- **Achtung:** wegen $\mathbf{B}_k \approx \mathbf{H}_f^{-1}$ ist die *Matrixgleichung* der **Kehrwert** des Differenzenquotienten (in 1D: $b_{k+1} = s_k/y_k$ statt y_k/s_k)
- Setze $\mathbf{B}_0 = \mathbf{I}$
- BFGS-Update mit $\rho_k = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k} \left(\frac{\mathbf{y}_k^\top \mathbf{s}_k}{\mathbf{s}_k^\top \mathbf{s}_k} \approx \text{Krümmung in Richtung } \mathbf{s}_k \right)$:

$$\mathbf{B}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{B}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

- Komplexität: $O(n^2)$ pro Schritt statt $O(n^3)$ bei Newton
- Konvergenz schneller als GD, fast so gut wie Newton (superlinear)
- **L-BFGS:** *Limited-memory* Variante für großes n — speichert nur letzte m Vektoren für Speicherkomplexität $O(mn)$ statt $O(n^2)$

Methodenvergleich: Zusammenfassung

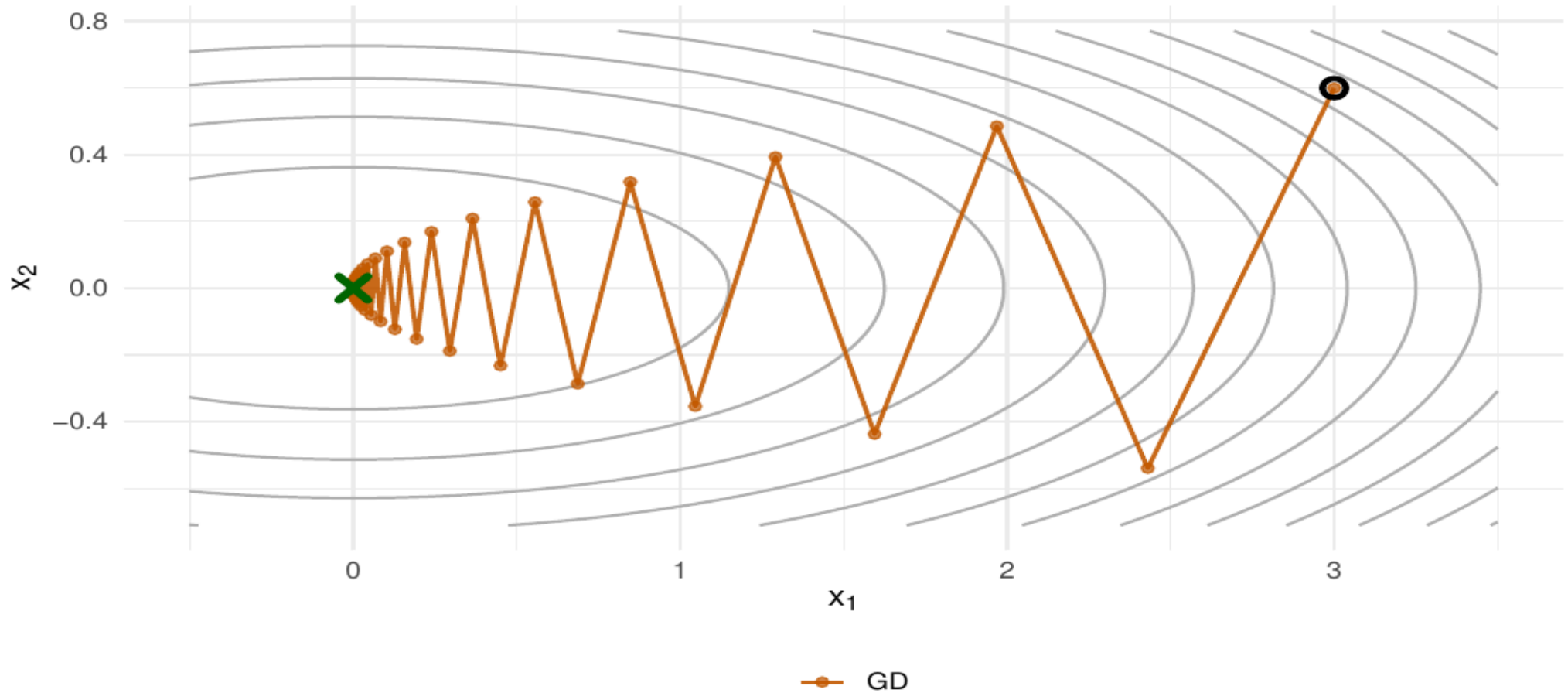
Eigenschaft	Nelder-Mead	Gradientenabstieg	Quasi-Newton	Newton
Verfahren	0. Ordnung	1. Ordnung	“1.5te” Ordnung	2. Ordnung
Benötigt	nur f	$f, \nabla f$	$f, \nabla f$	$f, \nabla f, \mathbf{H}_f$
Komplexität/Schritt	$O(n)$	$O(n)$	$O(n^2)$	$O(n^3)$
Konvergenz	Keine Garantie	Linear [×]	Superlinear [×]	Quadratisch [×]
Schrittweite	Autom.	Kritisch (γ)	<i>line search</i>	Autom.
Skalierbarkeit	Schlecht ($n \leq 10$)	Sehr gut	Gut	Schlecht

×: Konvergenzaussagen gelten lokal bzw. für glatte, stark konvexe Zielfunktionen und geeignete Schrittweiten/Line Search.

- GD/Newton/Quasi-Newton sind **Fixpunktiterationen** für $\nabla f(\mathbf{x}) = \mathbf{0}$
- **Nelder-Mead**: Default in `R` `optim()`; nützlich, wenn keine Ableitungen verfügbar
- **GD**: Grundlage moderner NN-Optimierer
- **Quasi-Newton (BFGS)**: in `R`: `optim(..., method = "BFGS")`, guter Kompromiss
- **Newton**: MaxLik-Inferenz (IWLS für GLMs), XGBoost, ...

Momentum & Stochastic Gradient Descent

Gradientenabstieg mit Momentum: Motivation



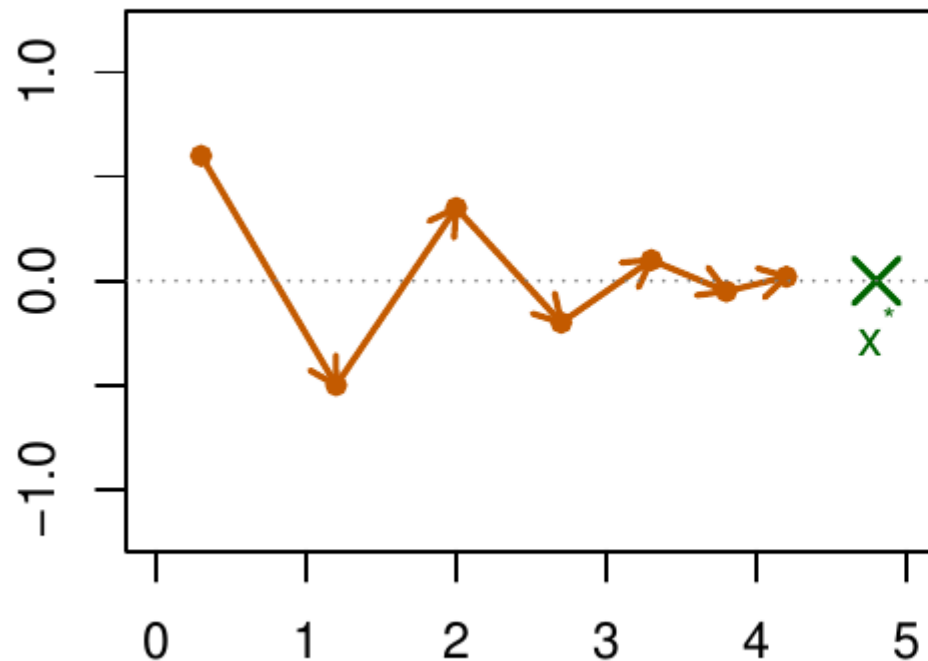
Bei **hoher Konditionszahl** $\kappa(\mathbf{H}_f)$ oszilliert GD oft stark.

Idee: Momentum ("Schwung")

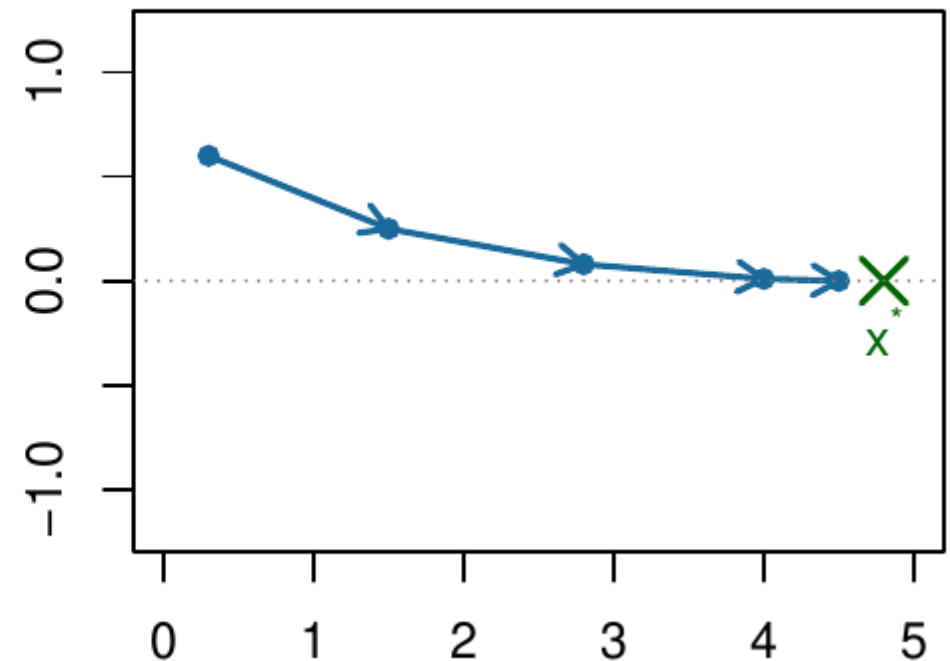
Analogie:

Eine Kugel, die einen Hügel herunterrollt, sammelt Schwung (*momentum*). Ihre Bewegung wird dann nicht durch jede kleine Unebenheit beeinflusst, sondern durch die *mittlere Steigung*.

Ohne Momentum



Mit Momentum



Momentum dämpft Oszillationen und beschleunigt konsistente Richtungen.

Heavy-Ball Momentum

Momentum-Update

Benutzt wird nicht mehr nur die aktuelle Steigung $\nabla f(\mathbf{x}^{(k)})^\top$, sondern ein speziell “gemittelter” Gradient $\mathbf{v}^{(k)}$:

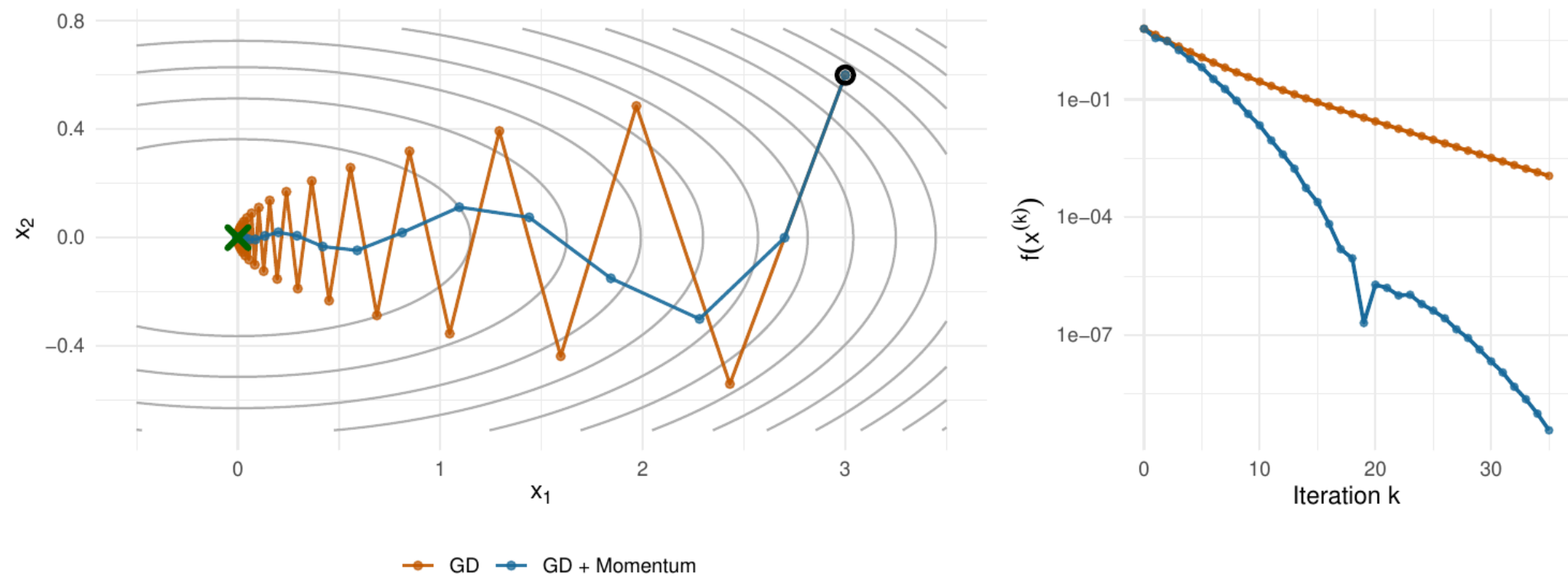
$$\begin{aligned}\mathbf{v}^{(k+1)} &= \alpha \mathbf{v}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}\end{aligned}$$

- $\mathbf{v}^{(k)}$: “Geschwindigkeit” (akkumulierte Bewegungsrichtung)
- $\alpha \in [0, 1)$: **Momentum**-Parameter (typisch: 0.9); gedämpft wird mit $1 - \alpha$
- γ : Schrittweite (wie bei GD)
- (Bessere) Alternative: **Nesterov-Momentum** benutzt Gradient an *voraussichtlichem* Punkt.

Wirkung:

- Glättet Oszillationen durch Mittelung früherer Gradienten
- Beschleunigt so Iterationsfortschritt in Richtungen mit konsistentem Gradienten

Momentum: Visualisierung



s.a. Shiny-App

Stochastic Gradient Descent: Motivation

Typisches ML/Statistik-Problem:

Minimiere das *empirische Risiko*

$$R(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^N L(y_i, p_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

- Modell $p_{\boldsymbol{\theta}}$ mit Features/Inputs $\mathbf{x}_i$, Labels/Outputs y_i und Parametern $\boldsymbol{\theta}$
- N = Anzahl Datenpunkte (kann sehr groß sein!)
- Verlustfunktion $L(y_i, p_{\boldsymbol{\theta}}(\mathbf{x}_i)) \simeq$ Fehler für Datenpunkt i bei Modellvorhersage $\hat{y}_i = p_{\boldsymbol{\theta}}(\mathbf{x}_i)$

Problem:

$$\nabla R(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \nabla L(y_i, p_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

Jeder Schritt erfordert Berechnung **aller** N Gradienten $\rightarrow$ **sehr teuer!**

Stochastic Gradient Descent (SGD)

Idee:

Gradient eines zufälligen Datenpunkts $\nabla L(y_i, p_{\theta}(\mathbf{x}_i))$ ist ein *unverzerrter Schätzer*:

$$\mathbb{E}_{i \sim \text{Unif}\{1, \dots, N\}} [\nabla L(y_i, p_{\theta}(\mathbf{x}_i))] = \nabla R(\theta)$$

(Erwartungswert *nur* über die Ziehung des Index i — bei festem Datensatz und festem θ)

SGD-Update

Wähle zufälligen Index i_k :

$$\theta^{(k+1)} = \theta^{(k)} - \gamma^{(k)} \nabla L(y_{i_k}, p_{\theta^{(k)}}(\mathbf{x}_{i_k}))^\top$$

- Kosten pro Schritt: $O(1)$ statt $O(N)$
- Richtung ist “im Mittel” korrekt, aber **verrauscht**

In der Praxis:

Verwende **Mini-Batches**: Gradient bestimmt aus Stichproben von $n \ll N$ Datenpunkten (oft: 32–256)

SGD: Grundlage moderner AI

Mini-Batch-Gradientenverfahren sind die Grundlage fast des gesamten aktuellen NN-Trainings.

- Zwei langlebige Standardfamilien:
 - **SGD mit Momentum** (ggf. Nesterov), besonders in Computer Vision
 - **AdamW**: adaptive erste und zweite Momente + entkoppelter *Weight Decay*; Standard bei Transformern
- Adaptive, veränderliche Schrittweiten: **Learning-Rate-Schedule** oft ebenso wichtig wie der Optimierer (*Warmup* → lineare oder cos-Abnahme.)
- **Gradient Clipping** begrenzt große Gradienten und stabilisiert das Training.
- Mini-Batch-Rauschen kann helfen, Sattelpunkte und Plateaus zu verlassen.

Weiterführend:

MSc-Kurs **Optimization for ML** behandelt Adam, Muon & Co, *learning rate schedules*, etc.

Beschränkte Optimierung (Constrained Optimization)

Optimierung mit Nebenbedingungen: Motivation

Bisher: **Unbeschränkte** Optimierung $\min_{\mathbf{x}} f(\mathbf{x})$ für $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

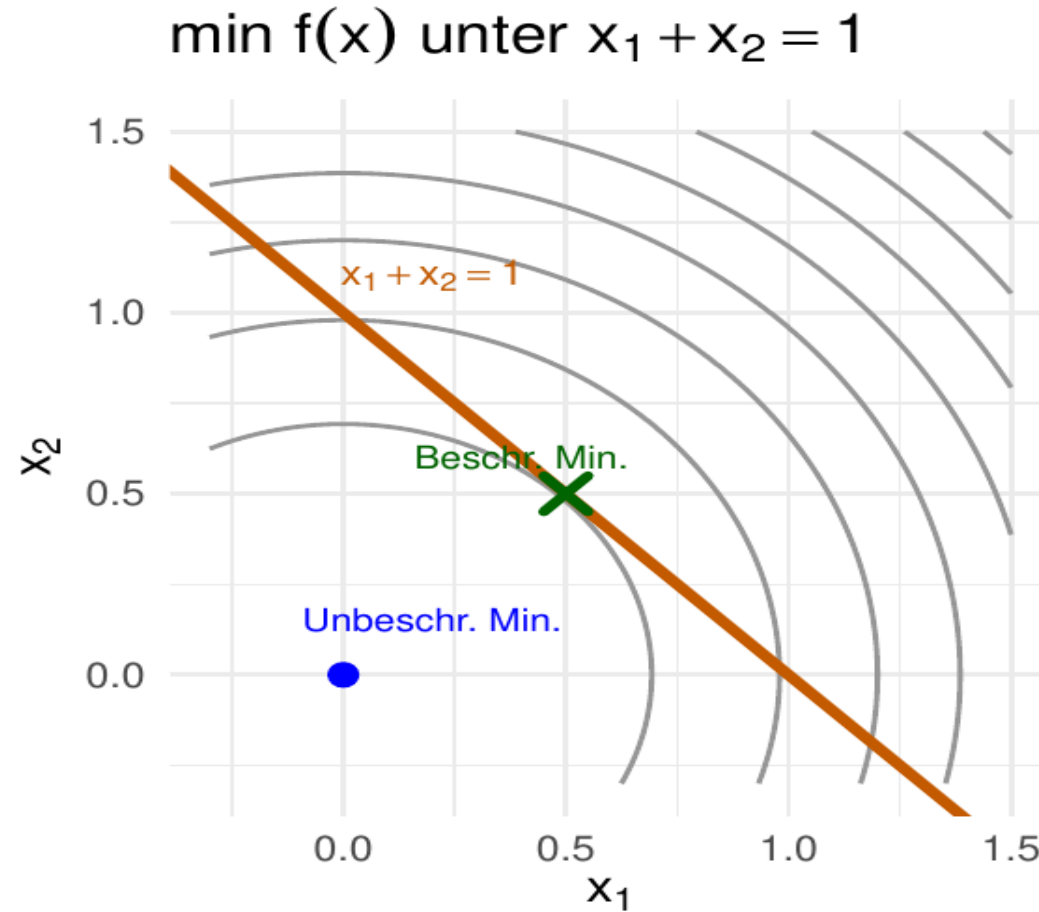
Oft haben wir aber **Einschränkungen**, z.B.:

- Parameter sind Wahrscheinlichkeiten: $p_i \geq 0$ und $\sum_i p_i = 1$
- Budget: Kosten $\leq$ Maximum
- Physik: Erhaltungsgrößen

Allgemeines Problem mit Nebenbedingungen

$\min_{\mathbf{x}}$	$f(\mathbf{x})$	(Zielfunktion)
u.d.N.	$g_i(\mathbf{x}) = 0, \ i = 1, \dots, m$	(Gleichungen)
	$h_j(\mathbf{x}) \leq 0, \ j = 1, \dots, p$	(Ungleichungen)

Geometrische Intuition



Hier also: $g(x) = x_1 + x_2 - 1$. Das beschränkte Optimum liegt **auf** der Nebenbedingung!

Lagrange-Multiplikatoren: Idee

Intuition:

1. Wenn wir die Nebenbedingung $g(\boldsymbol{x}) = 0$ als Kurve betrachten, dürfen wir uns nur entlang dieser Kurve bewegen.
2. Im Optimum gibt es entlang der Nebenbedingung g keine Steigung mehr in f
 - $\implies \nabla f$ senkrecht zur (Tangente der) Nebenbedingung
 - $\implies \nabla f$ parallel zu ∇g

Idee: Am Optimum $\boldsymbol{x}^*$ auf $g(\boldsymbol{x}) = 0$ muss also gelten:

$$\nabla f(\boldsymbol{x}^*) \text{ ist parallel zu } \nabla g(\boldsymbol{x}^*)$$

$$\implies \exists \lambda \in \mathbb{R} \text{ mit } \nabla f(\boldsymbol{x}^*) + \lambda \nabla g(\boldsymbol{x}^*) = \mathbf{0}^\top$$

Lagrange-Multiplikatoren: Theorie

Lagrange-Funktion

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{x})$$

$$\text{mit } \mathbf{g}(\mathbf{x}) = \begin{pmatrix} g_1(\mathbf{x}) \\ \vdots \\ g_m(\mathbf{x}) \end{pmatrix} \text{ und } \boldsymbol{\lambda} = \begin{pmatrix} \lambda_1 \\ \vdots \\ \lambda_m \end{pmatrix}$$

Notwendige Bedingung (Lagrange)

Sind die $\nabla g_i(\mathbf{x}^*)$ linear unabhängig, existiert am Optimum $\mathbf{x}^*$ ein $\boldsymbol{\lambda}^*$ mit:

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0}^\top \quad \text{und} \quad \mathbf{g}(\mathbf{x}^*) = \mathbf{0}$$

- Hinreichende Bedingungen sind komplexer (siehe z.B. Heath, Ch. 6.2.3)

Beispiel: Lagrange-Multiplikatoren

Problem:

Minimiere $f(x, y) = x^2 + y^2$ unter $g(x, y) = x + y - 1 = 0$.

Lagrange-Funktion:

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Notwendige Bedingungen:

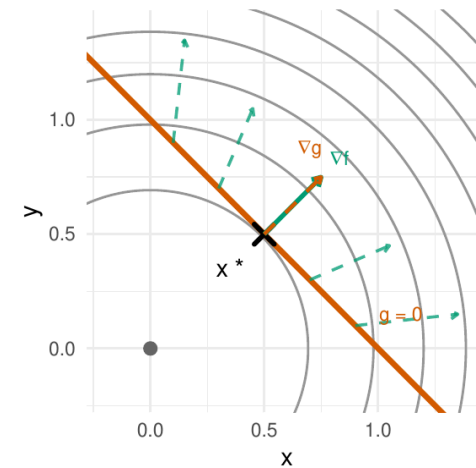
$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0 \quad \Rightarrow \quad x = -\lambda/2$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0 \quad \Rightarrow \quad y = -\lambda/2$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = g(x, y) = x + y - 1 = 0 \quad \Rightarrow \quad -\lambda = 1$$

$$\text{Lösung: } x^* = y^* = \frac{1}{2}, \lambda^* = -1, f(x^*, y^*) = \frac{1}{2}$$

Am Optimum: $\nabla f = -\lambda \nabla g$



$$\nabla f(x, y) = (2x, 2y); \quad \nabla g(x, y) = (1, 1)$$

KKT-Bedingungen für Ungleichungen

Bei **Ungleichungs-Nebenbedingungen** $h_j(\mathbf{x}) \leq 0$ wird es komplizierter.

Karush-Kuhn-Tucker (KKT) Bedingungen

Für das Problem

$$\min f(\mathbf{x}) \text{ u.d.N. } g_i(\mathbf{x}) = 0, h_j(\mathbf{x}) \leq 0$$

gelten am Optimum $\mathbf{x}^*$:

1. Stationarität: $\nabla f(\mathbf{x}^*) + \sum_i \lambda_i \nabla g_i(\mathbf{x}^*) + \sum_j \mu_j \nabla h_j(\mathbf{x}^*) = \mathbf{0}^\top$
2. Primale Zulässigkeit: $g_i(\mathbf{x}^*) = 0, h_j(\mathbf{x}^*) \leq 0$
3. Duale Zulässigkeit: $\mu_j \geq 0$
4. Komplementarität: $\mu_j h_j(\mathbf{x}^*) = 0$ für alle j

Komplementarität bedeutet:

Für jede Ungleichung gilt

- $h_j(\mathbf{x}^*) < 0$ (inaktiv) $\implies \mu_j = 0$
- $\mu_j > 0 \implies h_j(\mathbf{x}^*) = 0$ (aktiv, "bindet")

Beispiel: Ungleichungs-Nebenbedingungen

Problem: $\min_x f(x) = (x - 3)^2$ unter $0 \leq x \leq 2$, umformuliert: $h_1(x) = -x \leq 0$, $h_2(x) = x - 2 \leq 0$

KKT-Bedingungen:

1. $2(x^* - 3) - \mu_1 + \mu_2 = 0$
2. $-x^* \leq 0$, $x^* - 2 \leq 0$
3. $\mu_1, \mu_2 \geq 0$
4. $\mu_1 x^* = 0$, $\mu_2(x^* - 2) = 0$

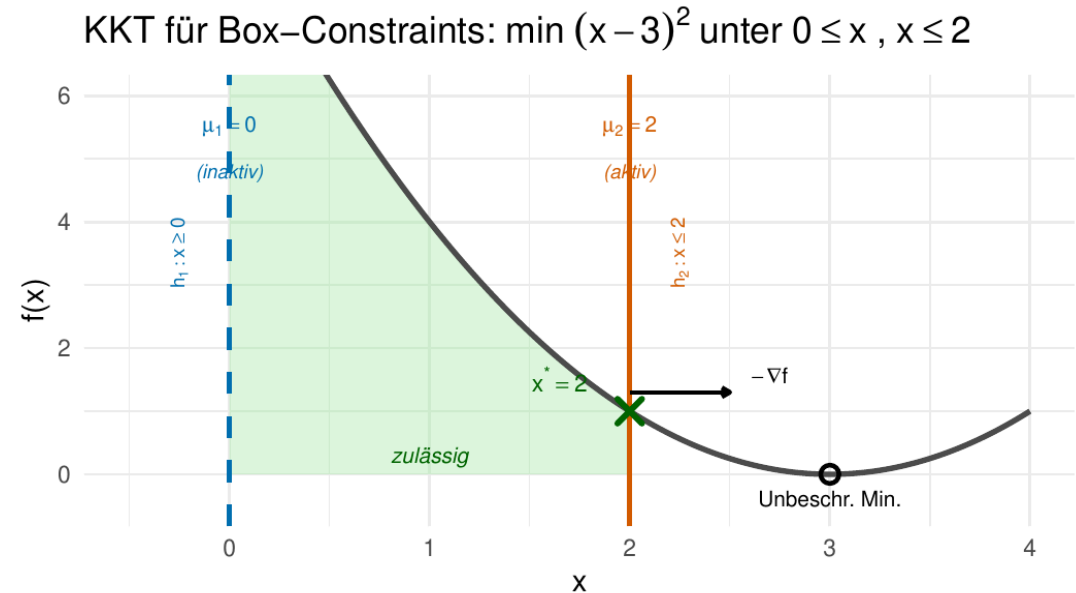
Lösung:

$x^* = 2$ (am rechten Rand), mit $\mu_1 = 0$, $\mu_2 = 2$

$\implies$ Obere Schranke ist **aktiv** (bindet), untere ist **inaktiv**.

Am Optimum $x^* = 2$:

neg. Gradient $-\nabla f(x^*) = -2(x^* - 3) = 2$ zeigt nach rechts (Richtung des unbeschränkten Minimums), aber die **aktive** Schranke h_2 blockiert weitere Bewegung.



Statistische Anwendung: Regularisierte Regression

Regularisierte Regression lässt sich als *beschränkte Optimierung* formulieren:

Ridge / Lasso (äquivalente Formulierungen)

$$\text{Penalisiert: } \min_{\beta} (\|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_p^p)$$

$$\text{Beschränkt: } \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 \quad \text{u.d.N.} \quad \|\beta\|_p^p \leq c$$

mit $p = 2$ (Ridge) bzw. $p = 1$ (Lasso).

Lagrange-Funktion:

$$\mathcal{L}(\beta, \mu) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \mu (\|\beta\|_p^p - c)$$

KKT-Stationarität für Ridge ($p = 2$):

$$\nabla_{\beta} \mathcal{L}^{\top} = -2\mathbf{X}^{\top}(\mathbf{y} - \mathbf{X}\beta) + 2\mu\beta = \mathbf{0} \implies \hat{\beta}_{\text{Ridge}} = (\mathbf{X}^{\top}\mathbf{X} + \mu\mathbf{I})^{-1}\mathbf{X}^{\top}\mathbf{y}$$

Für jedes $\lambda > 0$ existiert ein $c > 0$, sodass beide Formulierungen dieselbe Lösung haben –
umgekehrt nur, solange die Nebenbedingung auch *bindet* (ist der KQ-Schätzer selbst zulässig, erzwingt die Komplementarität $\mu = 0$).
Die Zuordnung $\lambda \leftrightarrow c$ hängt von den Daten ab.

Exkurs: Subgradienten

Proposition

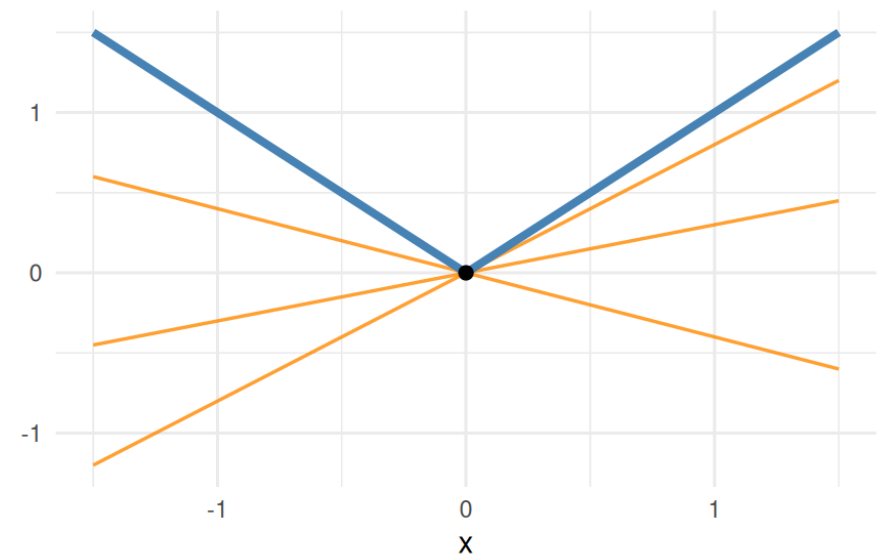
Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ konvex.

Dann gibt es für jedes $\mathbf{x} \in \mathbb{R}^n$ einen Vektor $\mathbf{v} \in \mathbb{R}^n$, sodass

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{v}^\top (\mathbf{y} - \mathbf{x}) \quad \forall \mathbf{y} \in \mathbb{R}^n.$$

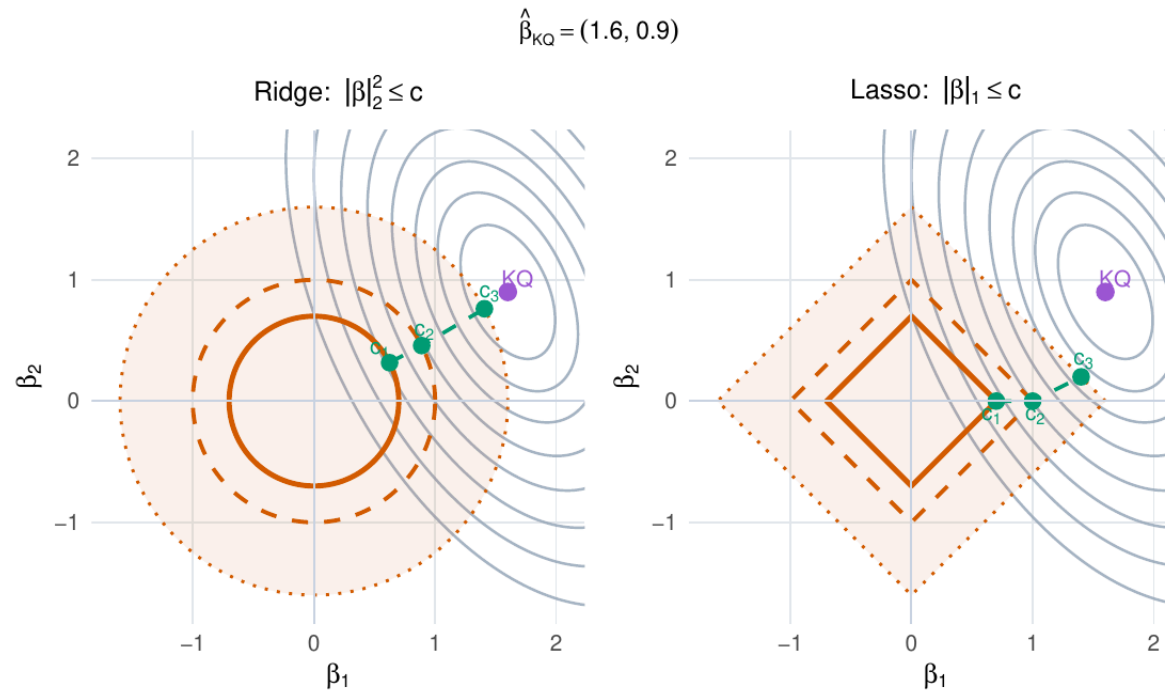
- Einfach gesagt: Wir können an jedem Punkt eine *affine* Funktion *unter* den Funktionsgraphen legen.
- Der Vektor $\mathbf{v}$ muss nicht eindeutig sein. *Jeder* Vektor, der diese Ungleichung erfüllt, wird **Subgradient** genannt. (Ist f in $\mathbf{x}$ differenzierbar, ist $\nabla f(\mathbf{x})^\top$ der *einzig*e Subgradient.)
- Die Funktion muss dafür **nicht differenzierbar** sein $\rightarrow$ **Subgradienten** wichtig für allgemeinere Optimierungsprobleme: z.B. LASSO, Quantilregression, u.v.m.

$f(x) = |x|$: Subgradienten am Knick $x = 0$



↩ zurück: „Geometrie: Ridge vs. Lasso“

Geometrie: Ridge vs. Lasso



Grau: Höhenlinien $f(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2$; Lila: $\arg \min f(\beta)$; Grün: Lösung unter Nebenbedingungen.

- **Ridge** ($p = 2$): Kreisförmige Nebenbedingung $\rightarrow$ Lösungen auf dem Kreisbogen (sofern die Nebenbedingung *bindet*)
- **Lasso** ($p = 1$): Rautenförmige Nebenbedingung $\rightarrow$ Lösungen oft an *Ecken*
 $\implies$ **sparse** ($\exists j : \beta_j = 0$)

Achtung:

$\|\beta\|_1$ ist an den Ecken **nicht differenzierbar** — die KKT-Stationarität gilt fürs Lasso nur mit **Subgradienten**.
In der Praxis verwendet man Subgradienten- bzw. *Proximal-Verfahren*.

Optimierung in R

Optimierung in R: `optimize`

Univariat: `optimize()` für $f : \mathbb{R} \rightarrow \mathbb{R}$

```
1 f <- function(x) (x - 3)^2
2 optimize(f, interval = c(0, 10))
```

```
$minimum
[1] 3
```

```
$objective
[1] 0
```

- Benutzt **Brent-Verfahren**: **Golden Section Search** (Intervall-Schrumpfung ähnlich **Bisektion**) + parabolische Interpolation
- Findet für **unimodales** f das Minimum im Intervall — sonst evtl. nur ein *lokales* Minimum

Optimierung in R: `optim`

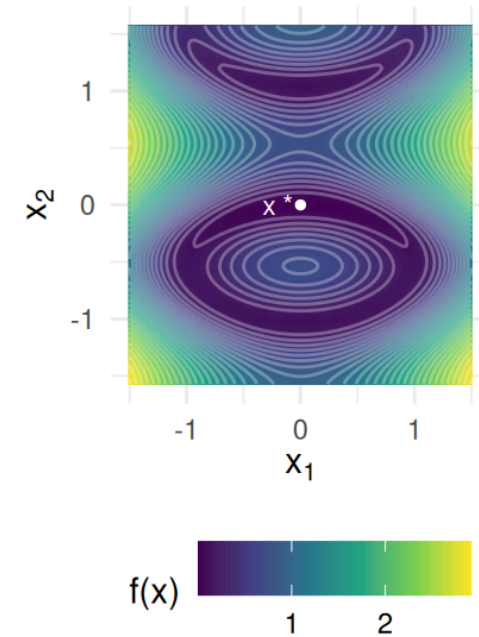
Multivariat: `optim()` für $f : \mathbb{R}^n \rightarrow \mathbb{R}$

```
1 f <- function(x) log1p((x[1]^2 + sin(3*x[2]))^2) +  
2   .1 * x[1]^2 + .1 * x[2]^2  
3 optim(c(-1, -0.5), f, method = "Nelder-Mead") |> #default  
4   str()
```

```
List of 5  
 $ par      : num [1:2] 5.54e-05 7.49e-06  
 $ value    : num 8.18e-10  
 $ counts   : Named int [1:2] 123 NA  
 .. attr(*, "names")= chr [1:2] "function" "gradient"  
 $ convergence: int 0  
 $ message  : NULL
```

```
1 optim(c(-1, -0.5), f, method = "BFGS") |> str()
```

```
List of 5  
 $ par      : num [1:2] 6.88e-10 2.11e-10  
 $ value    : num 4.52e-19  
 $ counts   : Named int [1:2] 47 29  
 .. attr(*, "names")= chr [1:2] "function" "gradient"  
 $ convergence: int 0  
 $ message  : NULL
```



optim: (Falsche) Konvergenz

Startwerte weit weg vom globalen Minimum $\mathbf{x}^* = (0, 0)$ bzw. zu kleines Iterationslimit:

```
1 # DANGER: Konvergenz zu lokalen Minima
2 optim(c(-1, 1), f)[1:2] |> str()
3 optim(c(-.5, -1), f, method = "BFGS")[1:2] |> str()
```

```
List of 2
 $ par   : num [1:2] 8.81e-05 1.04
 $ value : num 0.108
List of 2
 $ par   : num [1:2] -1.54e-05 -1.04
 $ value : num 0.108
```

```
1 # DANGER: Keine Konvergenz aber keine warnings():
2 optim(c(-1, -0.5), f, control = list(maxit = 50)) |> str()
```

```
List of 5
 $ par       : num [1:2] -0.584 -0.151
 $ value     : num 0.0454
 $ counts    : Named int [1:2] 51 NA
 ..- attr(*, "names")= chr [1:2] "function" "gradient"
 $ convergence: int 1
 $ message   : NULL
```

⇒ `convergence = 0` heißt nur: *Abbruchkriterium erfüllt* — **keine Garantie für ein (globales) Minimum!**

optim: Mit Gradienten oder Constraints

```
1 grad_f <- function(x) {  
2   log1p_dx <- 1 / (1 + (x[1]^2 + sin(3*x[2]))^2)  
3   c((4 * x[1] * (x[1]^2 + sin(3*x[2]))) * log1p_dx + .2 * x[1],  
4     (6 * cos(3*x[2]) * (x[1]^2 + sin(3*x[2]))) * log1p_dx + .2 * x[2])  
5 }  
6 str(optim(c(-1, -0.5), f, gr = grad_f, method = "BFGS")[-5])
```

```
List of 4  
$ par      : num [1:2] 6.88e-10 2.11e-10  
$ value    : num 4.52e-19  
$ counts   : Named int [1:2] 47 29  
..- attr(*, "names")= chr [1:2] "function" "gradient"  
$ convergence: int 0
```

```
1 str(optim(c(-1, 0.5), f, gr = grad_f, method = "L-BFGS-B",  
2         lower = c(-1, -1), upper = c(1, 1))[-5])
```

```
List of 4  
$ par      : num [1:2] 8.71e-07 2.65e-08  
$ value    : num 8.22e-14  
$ counts   : Named int [1:2] 14 14  
..- attr(*, "names")= chr [1:2] "function" "gradient"  
$ convergence: int 0
```

→ Analytischer Gradient schneller und genauer als numerische Approximation über finite Differenzen. → **L-BFGS-B** erlaubt Box-Nebenbedingungen (**KKT!**).

Self-Check & Zusammenfassung

Self-Check

Frage 1: Newton-Schritt für $f(x) = x^2 + 2$ mit $x^{(0)} = 4$: Was ist $x^{(1)}$?

1. $x^{(1)} = 0$
2. $x^{(1)} = 2$
3. $x^{(1)} = -4$
4. $x^{(1)} = 3.5$

Frage 2: Welche Aussage über die KKT-Bedingungen ist **falsch**?

1. Die Multiplikatoren für Ungleichungen erfüllen $\mu_j \geq 0$.
2. Komplementarität: $\mu_j h_j(\mathbf{x}^*) = 0$ für alle j .
3. Ist eine Ungleichung inaktiv ($h_j(\mathbf{x}^*) < 0$), gilt $\mu_j = 0$.
4. Ist eine Ungleichung inaktiv ($h_j(\mathbf{x}^*) < 0$), muss $\mu_j > 0$ gelten.

Frage 3: `optim()` liefert `convergence = 0`. Was wissen wir?

1. Das globale Minimum wurde gefunden.
2. Ein Abbruchkriterium wurde erfüllt — keine Garantie für ein (globales) Minimum.
3. Der Gradient ist exakt **0**.
4. Die Funktion ist konvex.

Wrap-up: Optimierung I & II

Methode	Wann verwenden?
Bisektion / Newton-Raphson	Nullstellen $f(x) = 0$, univariat
Nelder-Mead	Keine Ableitungen, niedrige Dimension ($n \leq 10$)
Gradient Descent	Große Probleme, einfache Implementierung
Momentum	GD + schlecht konditionierte Probleme
SGD	Sehr große Datensätze & große Dimension
Quasi-Newton (BFGS)	Mittlere Probleme, guter Kompromiss
Newton	Kleine Probleme, schnelle Konvergenz nötig
Lagrange/KKT	Nebenbedingungen

Kernideen der beiden Optimierungs-Sitzungen:

- Unbeschränkte differenzierbare Optimierung $\simeq$ Lösen von $\nabla f(x) = \mathbf{0}^\top$
 - meist **iterativ** mit - unter Nebenbedingungen: mit **Lagrange**, **KKT** prüfen.
- Mehr Ableitungsinformation $\rightarrow$ schnellere Konvergenz, teurere Schritte
- **Konvexität** garantiert globale Optima; **Konditionszahl** der **Hessematrix** steuert das Tempo
- Nebenbedingungen: **Lagrange** / **KKT** — Gradienten von Ziel und Nebenbedingung im Gleichgewicht

Nächste Vorlesung: Funktionsapproximation

Anhang

Momentum: Zusammenfassung

Wann Momentum verwenden?

- Hilfreich vor allem bei **schlecht konditionierten** Problemen; bei kleiner Konditionszahl kann $\alpha = 0.9$ sogar *langsamer* sein als reines GD
- Standard in Deep Learning ($\alpha = 0.9$ typisch)
- Kombination mit adaptiven Schrittweiten (Adagrad, RMSprop): **ADAM**, ...

Verbindung zur Konvexität:

- Bei **quadratischen** Zielfunktionen (und geeignet gewählten α, γ) beschleunigt Momentum die Konvergenz beweisbar auf die Rate $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$; für allgemeine stark konvexe f garantiert das erst die *Nesterov*-Variante
- Reduziert effektiv die “Konditionszahl” des Problems